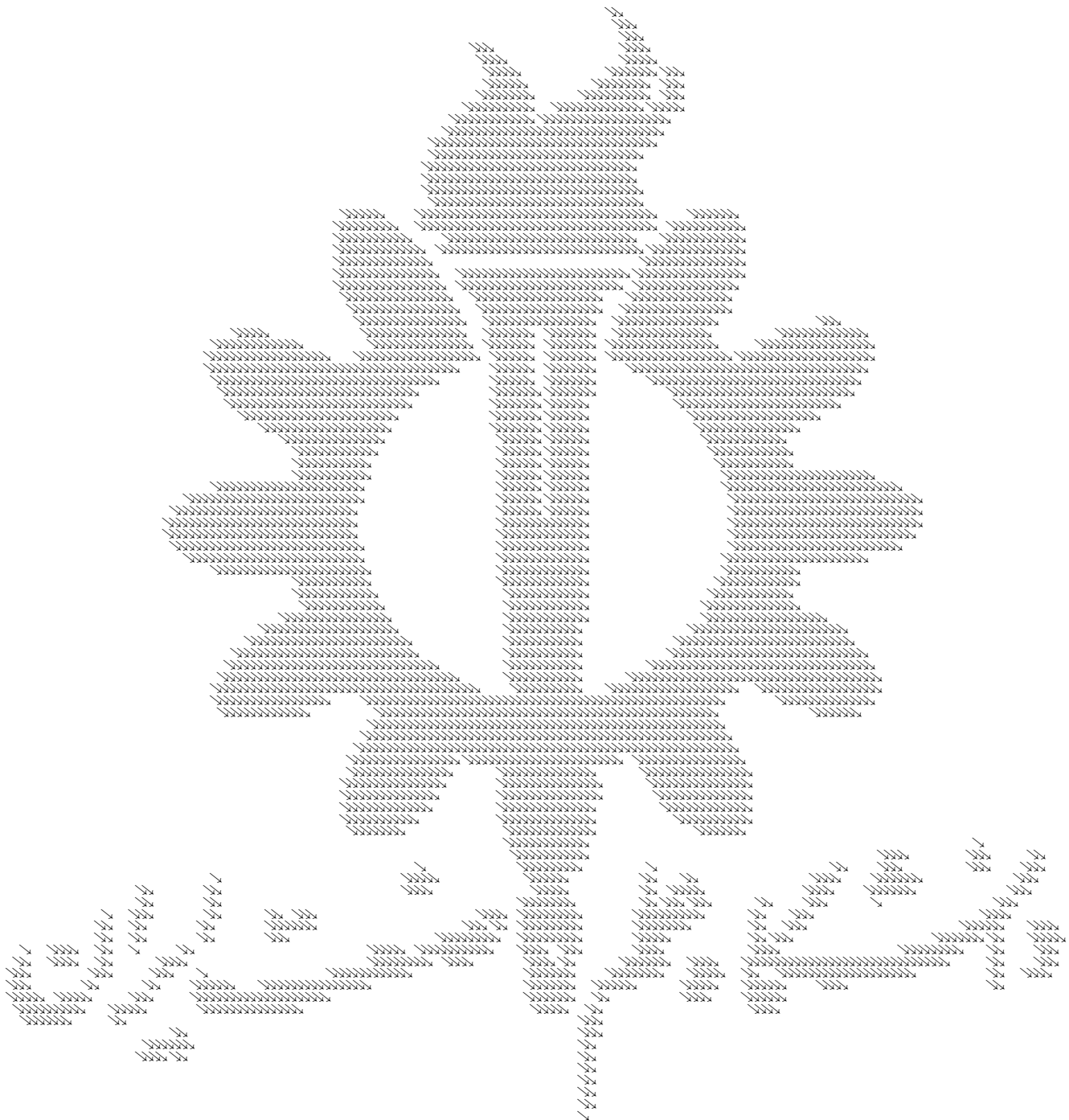




The Lost Ruins of Algorithmia

Design Algorithms — Final Project



Instructor: *Prof. Aghahadi*

Deadline: *21 Khordad 1405*



Contents

The Main Story	2
Problem 1 — “The Time-Limited Expedition”	3
Problem 2 — “Mapping the Ruins”	5
Problem 3 — “The Mandatory Collection”	7
Problem 4 — “The Rescue Mission”	9
Written Report	11
Grading	11
Submission Format	11
A Note on Using AI	12



The Main Story

Deep in the mountains of the ancient land of **Algorithmia**, archaeologists have discovered a vast underground ruin — a grid of stone chambers, each connected to its neighbours by narrow corridors. Many corridors have long since collapsed, becoming impassable walls. Others hide long-forgotten **treasure chests**, left behind by a civilisation that vanished centuries ago.

You are part of the expedition team. Your mission is to navigate this ruin, recover as many treasures as possible, and understand its structure — all while working with limited supplies and time. The ruin is represented as a **2D grid**:

```
. . # . T . .  
. T . # . . .  
# . . . . # .  
. . # T . . .  
. . . . # . .
```

Symbol	Meaning
.	Empty chamber — passable
#	Collapsed wall — impassable
T	Treasure chest
(any cell)	Can be a start S or end E per query

Movement rules:

- You may move **up, down, left, or right** — one chamber per move.
- Diagonal movement is **not allowed**.
- You **cannot** pass through # cells.
- Picking up a treasure is free — it is collected automatically when you enter its cell.
- You may revisit cells, but a treasure can only be collected **once**.

The ruin's grid and its treasures are fixed. What changes between missions is where you **start**, where you need to **end up**, and what **constraints** you face.



Problem 1 — “The Time-Limited Expedition”

Story

Your team enters the ruin at chamber **S**. The exit is at chamber **E**. You have enough supplies for at most **M moves**. You must exit through **E** — staying inside is not an option. Before you leave, you want to collect as many treasure chests as possible.

The question is: **given M moves, what is the maximum number of treasures you can collect starting at S and ending at E?**

Formal Definition

Given a start cell **S**, an end cell **E**, and a move budget **M**, find the **maximum number of treasures** collectible on any path from **S** to **E** that uses **at most M moves**.

If no path from **S** to **E** exists within **M moves**, output **-1**.

Input Format

```
R C
row_0
row_1
...
row_{R-1}
S_row S_col E_row E_col M
```

- **Line 1:** **R C** — number of rows and columns.
- **Lines 2 to R+1:** The grid rows, space-separated. Each cell is **.**, **#**, or **T**.
- **Last line:** Start position, end position, and move budget.

Output Format

A single integer: the maximum number of treasures collectible, or **-1** if no valid path exists.

Example

Input:

```
3 5
. T . . .
. # . T .
. . . . .
0 0 2 4 8
```

Output:

```
2
```

One optimal path: $(0,0) \rightarrow (0,1)[T] \rightarrow (0,2) \rightarrow (1,2) \rightarrow (1,3)[T] \rightarrow (2,3) \rightarrow (2,4)$ — 6 moves, 2 treasures.



Function Signature

```
def solve(grid: list[list[str]],  
          R: int, C: int,  
          sr: int, sc: int,  
          er: int, ec: int,  
          M: int) -> int:  
    ...
```

Returns the **maximum number of treasures** collectible on any path from (sr, sc) to (er, ec) using at most M moves, or -1 if no such path exists.

Algorithm: BFS / DP over a 4-dimensional state space.

State: $(row, col, moves_used, mask)$ where $mask$ is a bitmask of which treasures have been collected so far (index each T cell $0, 1, \dots$ in reading order).

Notes

- A path may revisit cells, but each treasure is collected at most once.
- You must **end exactly at E** (not just pass through it).
- If $S == E$ and $M \geq 0$, a path of 0 moves is valid (collecting the treasure at S if one exists).

Limits

Parameter	Constraint
$R \times C$	$R \times C \leq 100$
Number of treasures T	$0 \leq T \leq 10$
Move budget M	$1 \leq M \leq 200$



Problem 2 — “Mapping the Ruins”

Story

Back at base camp, the expedition’s cartographer wants to understand the ruin’s overall structure. Rescue teams may need to travel between **any two chambers** in an emergency — so knowing the average travel time across all possible (start, end) pairs is critical for planning supply caches and staffing.

Compute the **average minimum number of moves** to travel between any two reachable chambers in the ruin.

Formal Definition

Let P be the set of all passable cells (non-#). For two cells $u, v \in P$, let $d(u, v)$ be the length of the shortest path between them (∞ if unreachable).

Compute:

$$\text{average} = \frac{\sum_{\substack{(u,v): u \neq v \\ d(u,v) < \infty}} d(u, v)}{|\{(u, v) : u \neq v, d(u, v) < \infty\}|}$$

Output the result **truncated** to **2 decimal places**.

If no pair of distinct reachable cells exists, output 0.00.

Input Format

```
R C
row_0
row_1
...
row_{R-1}
```

- **Line 1:** $R \ C$ — number of rows and columns.
- **Lines 2 to $R+1$:** The grid rows, space-separated. Each cell is ., #, or T.

No query follows. The program reads only the grid.

Output Format

A single value: the average minimum distance truncated to 2 decimal places.

Example

Input:

```
2 3
. . #
. # .
```

Output:



1.33

(The exact average is $1.333\dots$, which truncates to 1.33.)

Function Signature

```
def solve(grid: list[list[str]],
          R: int, C: int) -> str:
    ...
```

Returns the average shortest distance over all **ordered** pairs of passable cells (u, v) with $u \neq v$ and $d(u, v) < \infty$, as a string **truncated** to 2 decimal places (e.g. "1.33"). Returns "0.00" if no valid pair exists.

Algorithm: Run one standard BFS from every passable cell. Accumulate a running **total** (sum of finite distances) and **count** (number of valid ordered pairs), then truncate $\text{total} / \text{count}$. Use $\text{int}(\text{result} * 100) / 100$ or `math.floor` to truncate without rounding.

Notes

- Only **ordered** pairs (u, v) with $u \neq v$ and $d(u, v) < \infty$ are counted.
- Cells that cannot reach any other cell are excluded from both the sum and the count.
- Truncation means dropping digits beyond the second decimal place without rounding. For example: $2.349 \rightarrow 2.34$, and $2.999 \rightarrow 2.99$.

Limits

Parameter	Constraint
R, C	$1 \leq R, C \leq 100$



Problem 3 — “The Mandatory Collection”

Story

A senior archaeologist insists that **at least K treasures** must be recovered from this expedition — anything less would not justify the cost. She doesn’t care how long it takes, but she wants the team to take the **shortest possible route** that still collects at least K treasures and ends at the designated exit E.

Find the **minimum number of moves** to go from S to E while collecting **at least K treasures** along the way.

Formal Definition

Given a start cell S, an end cell E, and a minimum requirement K, find the **minimum number of moves** for any path from S to E that collects **at least K treasures**.

If no such path exists, output -1.

Input Format

```
R C
row_0
row_1
...
row_{R-1}
S_row S_col E_row E_col K
```

- **Line 1:** R C — number of rows and columns.
- **Lines 2 to R+1:** The grid rows, space-separated. Each cell is ., #, or T.
- **Last line:** Start position, end position, and required treasure count.

Output Format

A single integer: the minimum number of moves, or -1 if no valid path exists.

Example

Input:

```
3 5
. T . . .
. # . T .
. . . . .
0 0 2 4 2
```

Output:

```
6
```

The shortest path from (0,0) to (2,4) that collects both treasures takes 6 moves.



Function Signature

```
def solve(grid: list[list[str]],
          R: int, C: int,
          sr: int, sc: int,
          er: int, ec: int,
          K: int) -> int:
    ...
```

Returns the **minimum number of moves** for any path from (sr, sc) to (er, ec) that collects **at least K treasures**, or -1 if no such path exists.

Algorithm: BFS over state $(row, col, mask)$, where $mask$ is a bitmask of collected treasures (indexed in reading order). Each state is visited **at most once**; the first time BFS reaches it is the minimum-move count.

Termination: Return the move count when BFS first dequeues a state where $(row, col) == (er, ec)$ and $\text{bin}(mask).count('1') \geq K$.

Notes

- If $K = 0$, this reduces to a standard shortest path problem.
- If the total number of treasures on the grid is less than K , output -1.

Limits

Parameter	Constraint
R, C	$1 \leq R, C \leq 100$
Number of treasures T	$0 \leq T \leq 10$
Required treasures K	$0 \leq K \leq T$



Problem 4 — “The Rescue Mission”

Story

A disaster has struck the ruin. Multiple rescue teams are stationed at chamber **S** and all need to reach chamber **E** as quickly as possible. However, the narrow corridors cannot fit two teams at once — **no two teams may occupy the same chamber at any point**, not even while passing through.

The question is: **what is the maximum number of teams that can simultaneously make the trip from S to E under this constraint?**

Formal Definition

Given a start cell **S** and an end cell **E**, find the **maximum number of vertex-disjoint paths** from **S** to **E** in the grid graph.

Two paths are **vertex-disjoint** if they share no cell other than **S** and **E** themselves.

Input Format

```
R C
row_0
row_1
...
row_{R-1}
S_row S_col E_row E_col
```

- **Line 1:** $R\ C$ — number of rows and columns.
- **Lines 2 to $R+1$:** The grid rows, space-separated. Each cell is `.`, `#`, or `T`. `T` cells are treated as passable in this problem.
- **Last line:** Start and end positions.

Output Format

A single integer: the maximum number of teams that can make the trip.

Example

Input:

```
3 3
. . .
. . .
. . .
0 0 2 2
```

Output:

```
2
```

Two vertex-disjoint paths: $(0,0) \rightarrow (0,1) \rightarrow (0,2) \rightarrow (1,2) \rightarrow (2,2)$ and $(0,0) \rightarrow (1,0) \rightarrow (2,0) \rightarrow (2,1) \rightarrow (2,2)$. *No third path exists that avoids both.*



Function Signature

```
def solve(grid: list[list[str]],
          R: int, C: int,
          sr: int, sc: int,
          er: int, ec: int) -> int:
    ...
```

Returns the **maximum number of vertex-disjoint paths** from (sr, sc) to (er, ec) .

Algorithm: Maximum flow on a *node-split* graph.

Graph construction: Split every passable cell (r, c) into two nodes $\text{node_in}(r, c)$ and $\text{node_out}(r, c)$ joined by an edge of capacity **1** (capacity ∞ for S and E). For each grid edge between adjacent passable cells u and v , add $\text{node_out}(u) \rightarrow \text{node_in}(v)$ with capacity ∞ (and the reverse direction). Run a max-flow algorithm (e.g. Edmonds–Karp) from $\text{node_out}(S)$ to $\text{node_in}(E)$.

Notes

- S and E themselves may be shared by all teams — only **intermediate** cells must be disjoint.
- If S and E are not connected, output 0.
- If $S == E$, output 1.

Limits

Parameter	Constraint
R, C	$1 \leq R, C \leq 200$



Written Report

Each problem must have a dedicated section in your report. For each problem, your report must cover:

- The algorithm you chose and why.
- The time and space complexity of your solution, with derivation.
- Any key design decision or insight that was central to your solution.

The report is **mandatory**. A correct solution submitted without a report section for that problem receives **0 points** for it.

Grading

There are 4 problems, each worth **25 points**, for a total of **100 points**. Each problem's 25 points are split between the code and the report:

Component	Points per problem
Correctness (code)	18 points
Written report	7 points

A test case earns full points only if the output is exactly correct and the solution finishes within **5 seconds**.

Benchmark (bonus, up to +20 points)

Among all solutions that pass correctness, a stress test will be run on large inputs, measuring total runtime. Bonus points are awarded per problem independently:

Rank within correct solutions	Bonus
Top 10%	+20
Top 25%	+16
Top 50%	+12
Top 75%	+8
Bottom 25% (but correct)	+4

Submission Format

```
student_id/  
|-- p1.py (or p1.cpp)  
|-- p2.py (or p2.cpp)  
|-- p3.py (or p3.cpp)  
|-- p4.py (or p4.cpp)  
'-- report.pdf
```

- Each problem must be in its **own separate file**.
- Accepted languages: **Python 3** and **C++17**.
- Each program reads from **stdin** and writes to **stdout**.



- No external libraries beyond the standard library are allowed.

Running your solution:

```
# Python
python3 p1.py < input.txt

# C++
g++ -O2 -std=c++17 -o p1 p1.cpp
./p1 < input.txt
```

A Note on Using AI

We know that accessing the internet in Iran has become harder than it used to be, and that reaching AI tools takes extra effort. But if you can — use them.

Not to solve the problems for you. Not as a shortcut. Use AI the way you would use a knowledgeable friend: to talk through your thinking, challenge your assumptions, and explore whether the approach you have in mind actually holds up. The ideas should come from you; AI is there to help you stress-test them.

Thinking is still your job. AI is just allowed to sit next to you while you do it.

Good luck, explorer. The ruins of Algorithmia await.